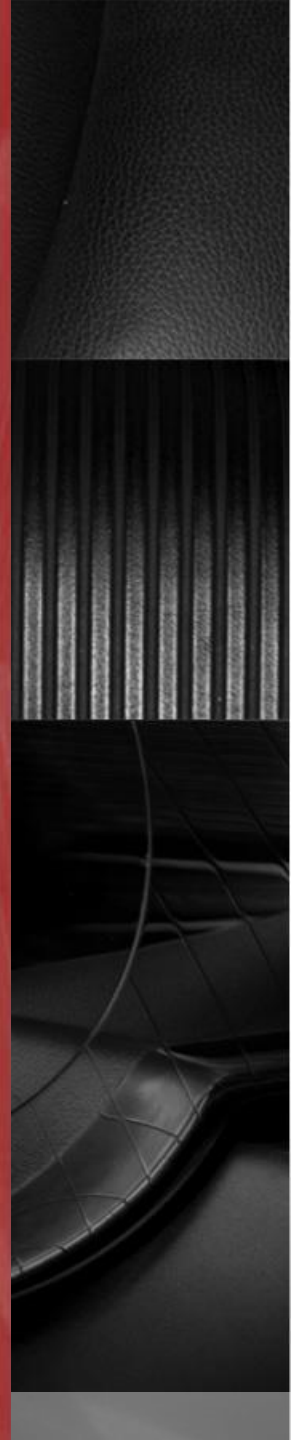
Bases de Datos

Grado en Ingeniería Informática

Tema 5. Diseño de bases de datos



Introducción

- Metodologías de diseño
  - Ciclo de vida de desarrollo de software
  - Ciclo de vida de desarrollo de bases de datos
- Diseño Conceptual
- Diseño Lógico
- Diseño Físico

Metodologías de diseño

- Una **metodología** es un conjunto de procedimientos, técnicas y ayudas a la documentación para el desarrollo de un producto software; en nuestro caso, el producto software es una BD.
- Una metodología nos indica:
  - las **actividades** a seguir en el desarrollo de la BD,
  - qué es **lo que hay que realizar** en cada actividad, indicando qué se necesita como entrada, qué se produce como salida e incluso quién está involucrado.

Metodologías de diseño

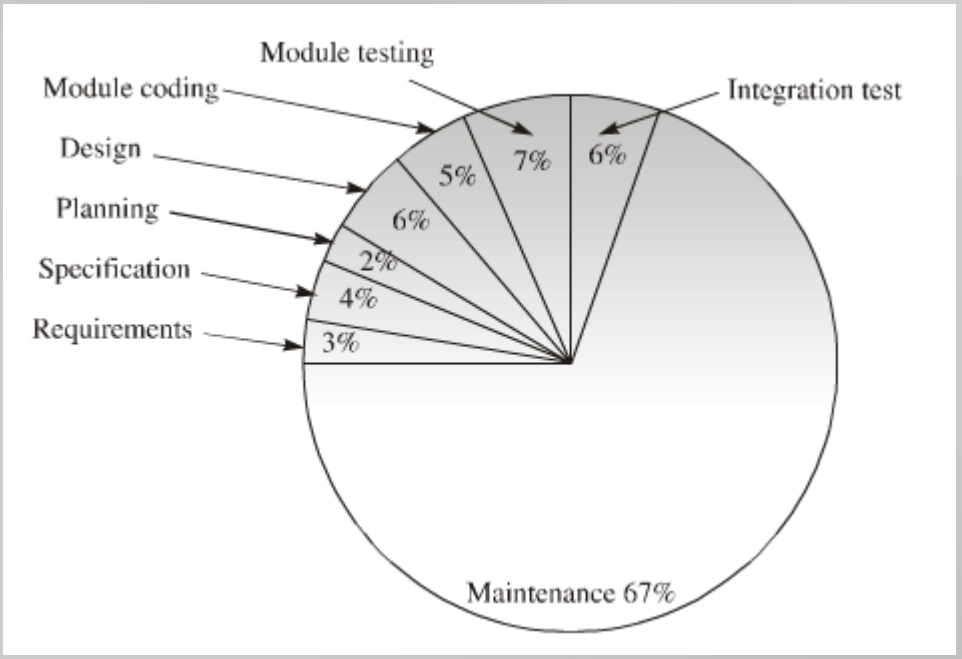
- Elementos que integran el proceso de diseño:
  - **Técnicas.** Representan cómo llevar a cabo cada una de las actividades o pasos de los que consta la metodología
    - **Procedimentales** (secuencia perfectamente definida de los pasos a realizar en una tarea como en un algoritmo)
    - **Heurísticas** (reglas, recomendaciones o sugerencias a seguir que en ningún caso establecen el proceso exacto de realización de una tarea)
  - **Modelos.** Instrumentos que empleamos para representar una determinada realidad. Los involucrados en el diseño de una base de datos son **modelos de datos.**
  - **Herramientas CASE.** Dan soporte automatizado para la utilización de las técnicas de una metodología así como a los modelos que incorporan

Metodologías de diseño

- Ciclo de vida de desarrollo de software
  - SDLC – Software Development Life Cycle
  - Metodología tradicional
  - Existen muchas alternativas
  - Hay una serie de procesos básicos que son comunes a todas las alternativas

Metodologías de diseño

- Ejemplo de SDLC
  - Requisitos
  - Especificación
  - Planificación
  - Diseño
  - Programación
  - Integración
  - Mantenimiento
  - Sustitución / Retirada

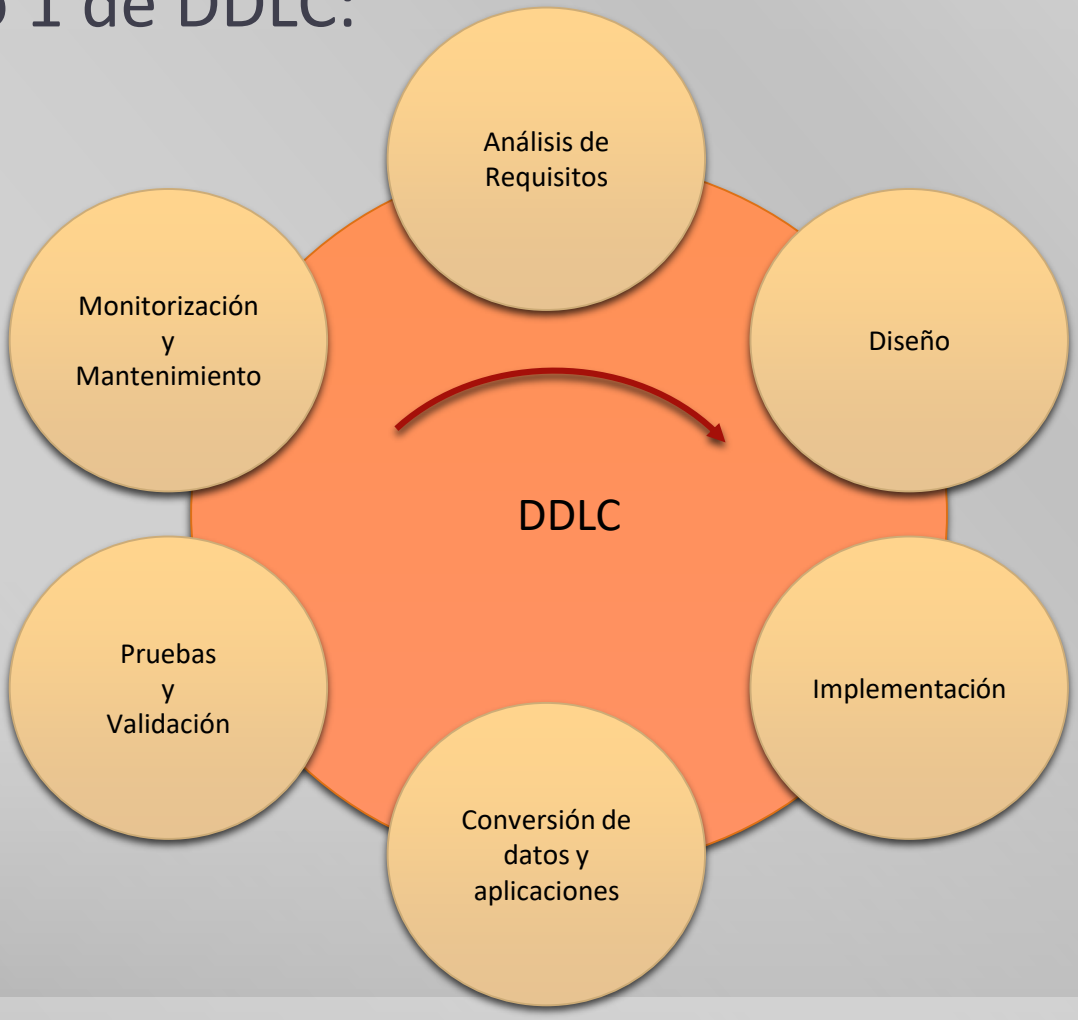


Metodologías de diseño

- Ciclo de vida de desarrollo de base de datos
  - DDLC – Database Development Life Cycle
  - Proceso para el diseño, implementación y mantenimiento de un sistema de bases de datos
  - Está directamente relacionado con el SDLC
  - El DDLC comienza en la fase de captura de requisitos del SDLC
  - De nuevo, existen varias alternativas, que tienen elementos comunes.

Metodologías de diseño

- Ejemplo 1 de DDLC:



Metodologías de diseño

- Ejemplo 1 de DDLC:
  - Análisis de Requisitos
    - Estudio de sistemas y procedimientos existentes
    - Determinación del ámbito y alcance de la base de datos
    - Captura de requisitos de información, hardware, software y procesamiento
    - Número y tipos de usuarios, y sus relaciones
    - Propiedades, restricciones y relaciones entre los datos
    - Datos, volumen de datos y necesidades de procesado
    - Requisitos operacionales
    - Ratio de crecimiento de la base de datos
    - Requisitos de seguridad

Metodologías de diseño

- Ejemplo 1 de DDLC:
  - Diseño de base de datos
    - Diseño conceptual
    - Diseño lógico
    - Diseño físico
    - Especificaciones de desarrollo
    - Ajuste de rendimiento de la base de datos

Metodologías de diseño

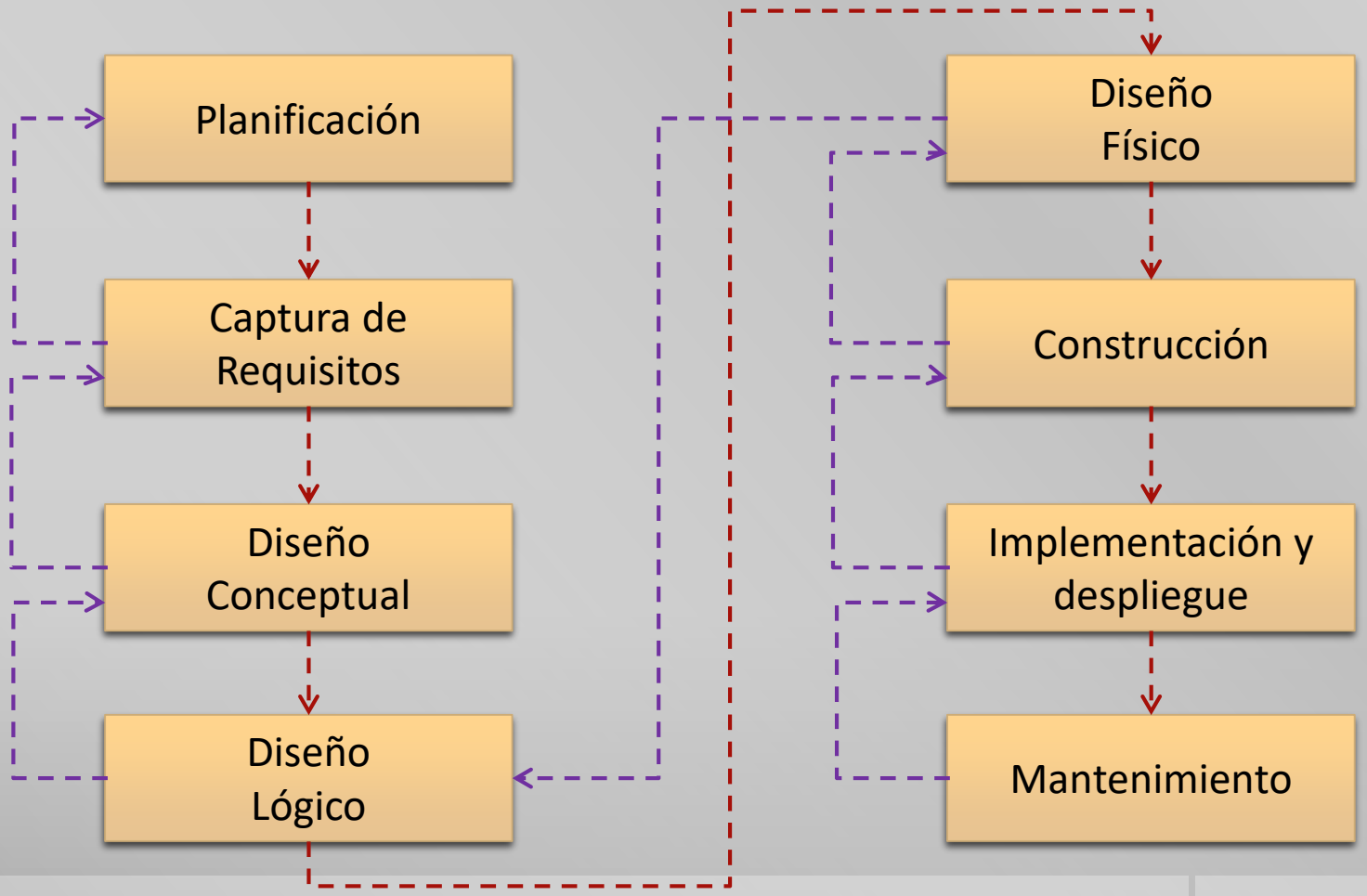
- Ejemplo 1 de DDLC:
  - Implementación de base de datos
    - Selección y adquisición del SGBD
    - Construcción del diccionario de datos
    - Creación de aplicaciones (software)
    - Desarrollo de procedimientos de base de datos
    - Entrenamiento de usuarios

Metodologías de diseño

- Ejemplo 1 de DDLC:
  - Conversión de datos y aplicaciones
    - Carga de datos en la base de datos
    - Migración de datos de aplicaciones anteriores
  - Pruebas y validación
  - Monitorización y mantenimiento

Metodologías de diseño

- Ejemplo 2 de DDLC:



Metodologías de diseño

- Ejemplo 2 de DDLC:
  - Planificación
    - Revisión de opciones del SGBD
    - Asignar especialista de BD al equipo de desarrollo
  - Requisitos
    - Recoger y analizar las perspectivas de los usuarios
    - Identificar entidades preliminares
  - Diseño Conceptual
    - Desarrollar el modelo de datos conceptual
  - Diseño Lógico
    - Desarrollar el modelo lógico
    - Normalización

Metodologías de diseño

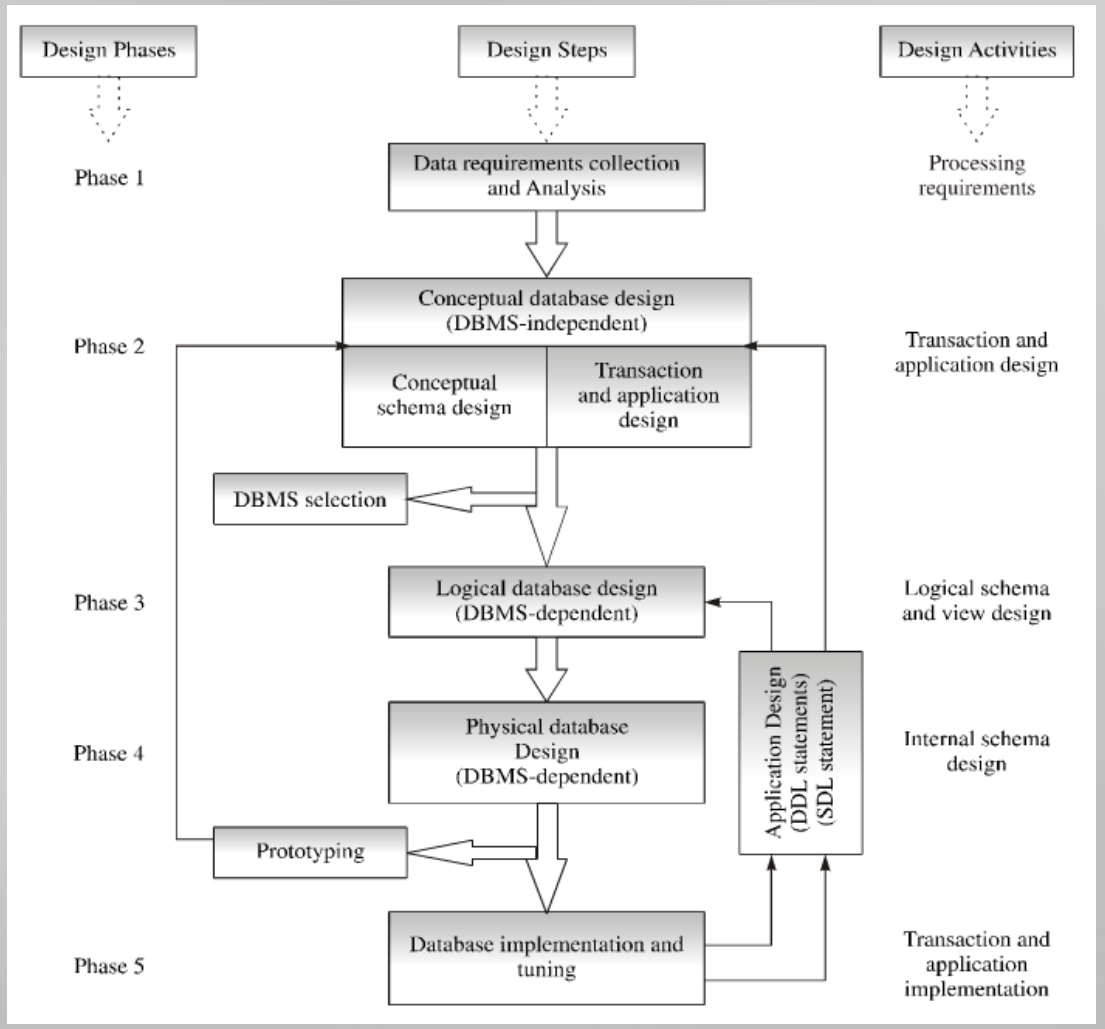
- Ejemplo 2 de DDLC:
  - Diseño Físico
    - Especificación de características finales de los datos
    - Ajuste para el rendimiento óptimo
  - Construcción
    - Creación de bases de datos de desarrollos y pruebas
    - Considerar necesidades de conversión de datos
  - Implementación
    - Creación de base de datos de producción
    - Realizar la conversión de datos
  - Mantenimiento
    - Análisis de rendimiento (tuning)
    - Actualización de software y parches
    - Implementar cambios derivados de modificaciones de aplicación

Metodologías de diseño

- Diseño de BD. Aproximaciones:
  - Bottom-up
    - Especificación de atributos, agrupación en entidades. Apropiado para pequeñas bases de datos.
  - Top-down
    - Definición de entidades y relaciones. Especificación de atributos. Definición de entidades y relaciones derivadas.
  - Inside-out
    - Determinación de entidades principales. Determinación de entidades relacionadas.
  - Mixed strategy
    - Combinación bottom-up y top-down.

Metodologías de diseño

- Diseño de BD



Diseño conceptual

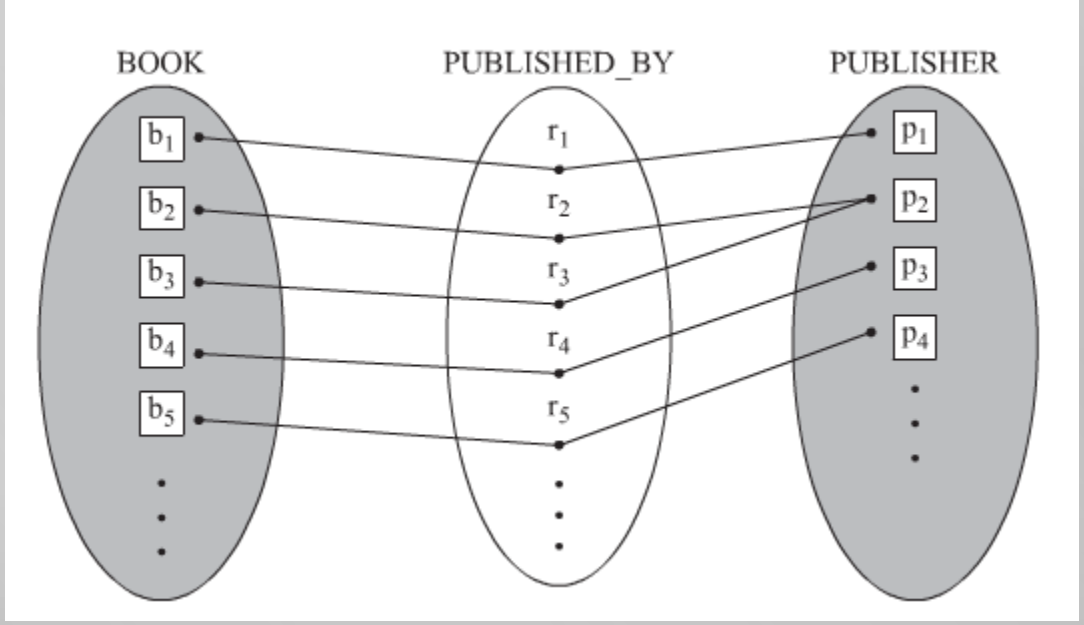
- Etapa del diseño que corresponde a la determinación del alcance y cobertura de la base de datos.
- También se le llama diseño semántico.
- Implica una definición de contenidos con un alto nivel de abstracción.
- Es una descripción concisa y permanente de la base de datos que permite la comunicación entre usuarios y desarrolladores.
- Es un modelado independiente del resto de procesos y decisiones que se tomen en ellos.
- Se apoya en el uso del modelo entidad-relación para su especificación.

Diseño conceptual

- Consideraciones sobre las entidades
  - Tipos e Instancias de entidades
  - Entidades regulares / débiles
- Consideraciones sobre atributos
  - Categorías:
    - Identificativos / Descriptivos
    - Uni-valuados / Multi-valuados
    - Almacenados / Derivados
    - Simples / Compuestos

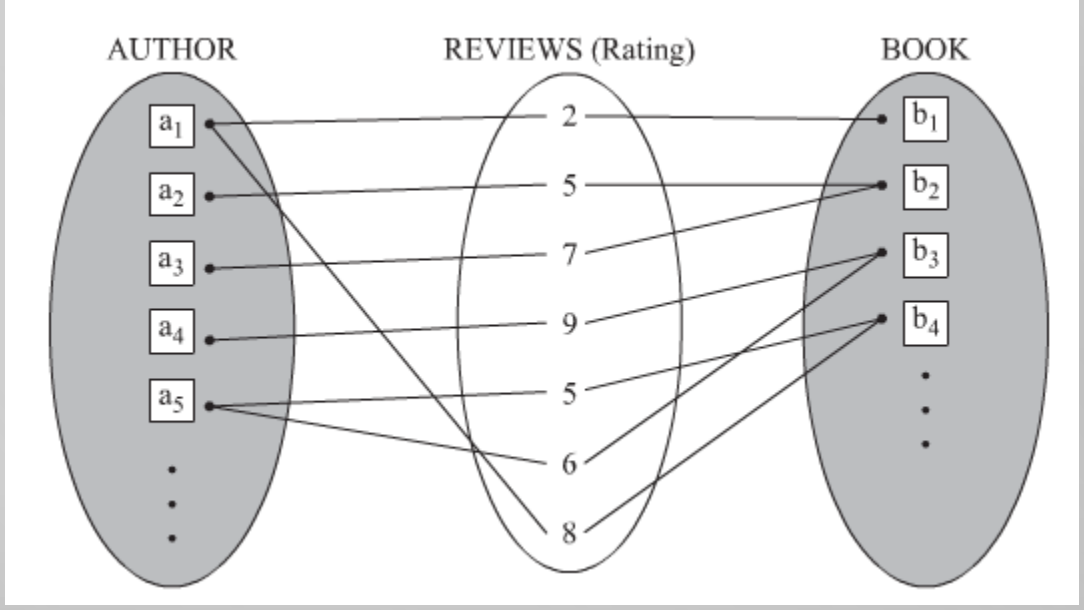
Diseño conceptual

- Consideraciones sobre relaciones:
  - Relaciones simples



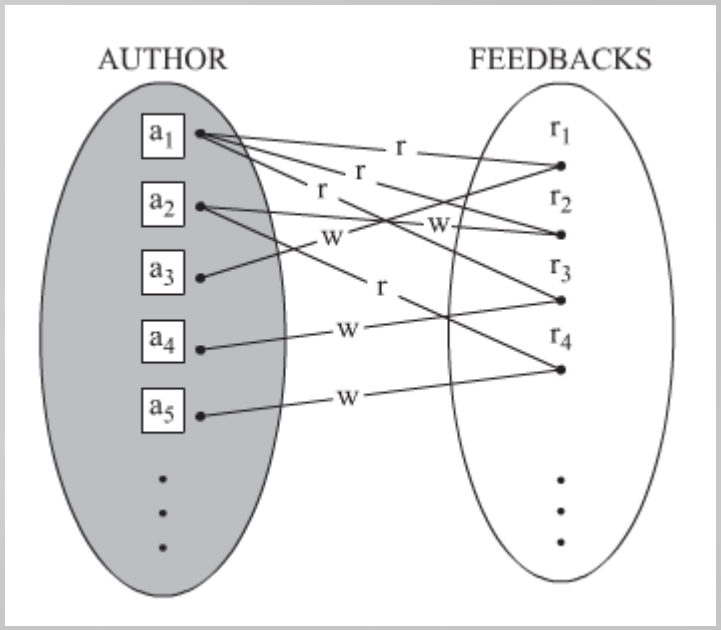
Diseño conceptual

- Consideraciones sobre relaciones:
  - Relaciones con atributos



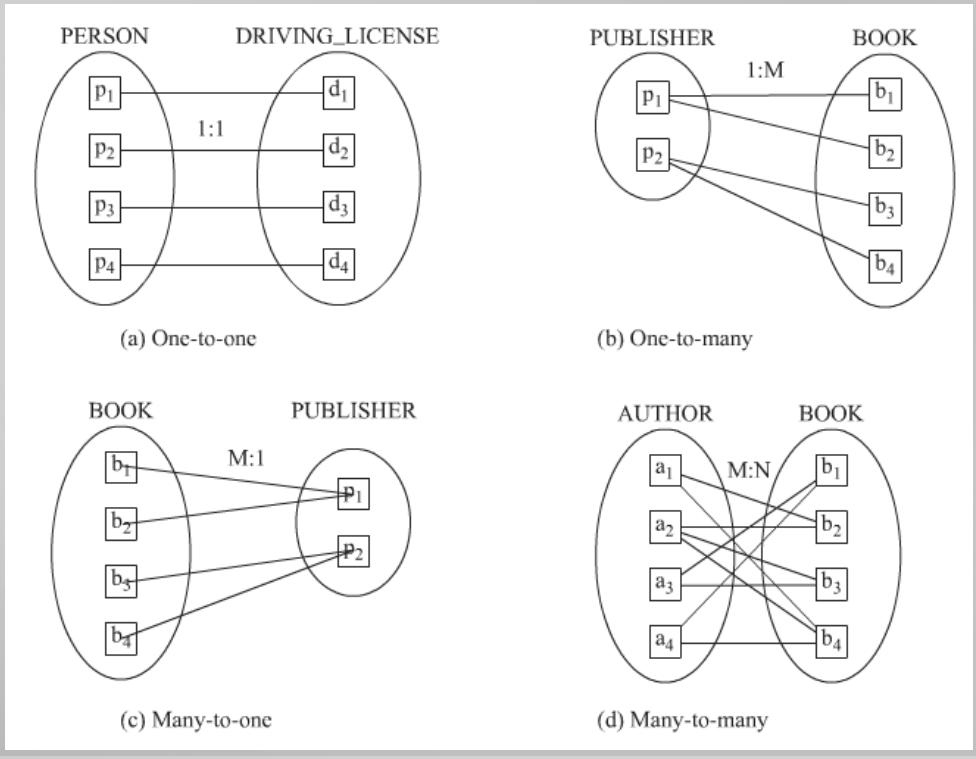
Diseño conceptual

- Consideraciones sobre relaciones:
  - Relaciones recursivas



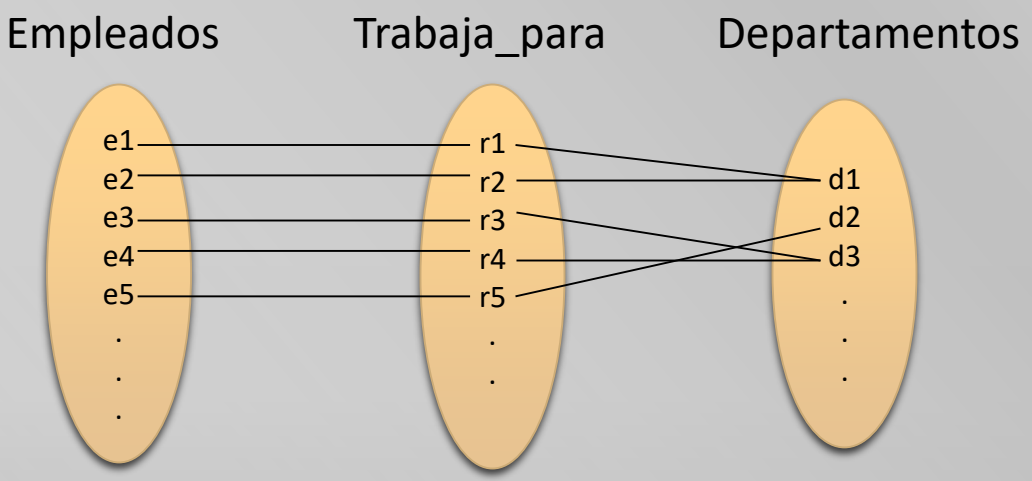
Diseño conceptual

- Consideraciones sobre relaciones:
  - Cardinalidades: one-to-one (1:1), one-to-many (1:M), many-to-one (M:1), many-to-many (N:M)



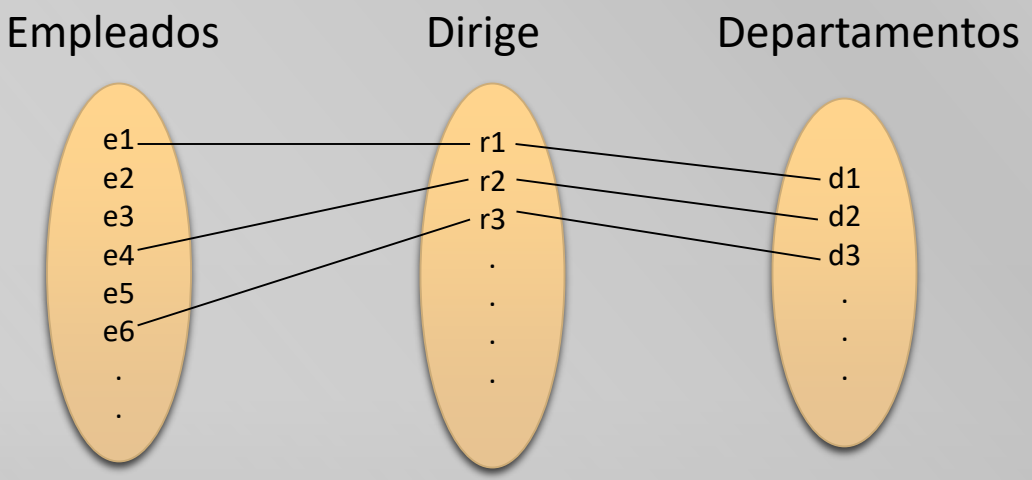
Diseño conceptual

- Consideraciones sobre relaciones:
  - Opciones de participación (cardinalidad mínima)
    - Participación total (obligatoria)



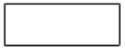
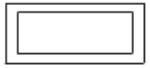
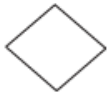
Diseño conceptual

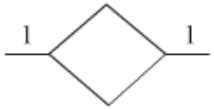
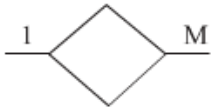
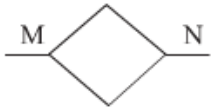
- Consideraciones sobre relaciones:
  - Opciones de participación (cardinalidad mínima)
    - Participación parcial (opcional)



Diseño conceptual

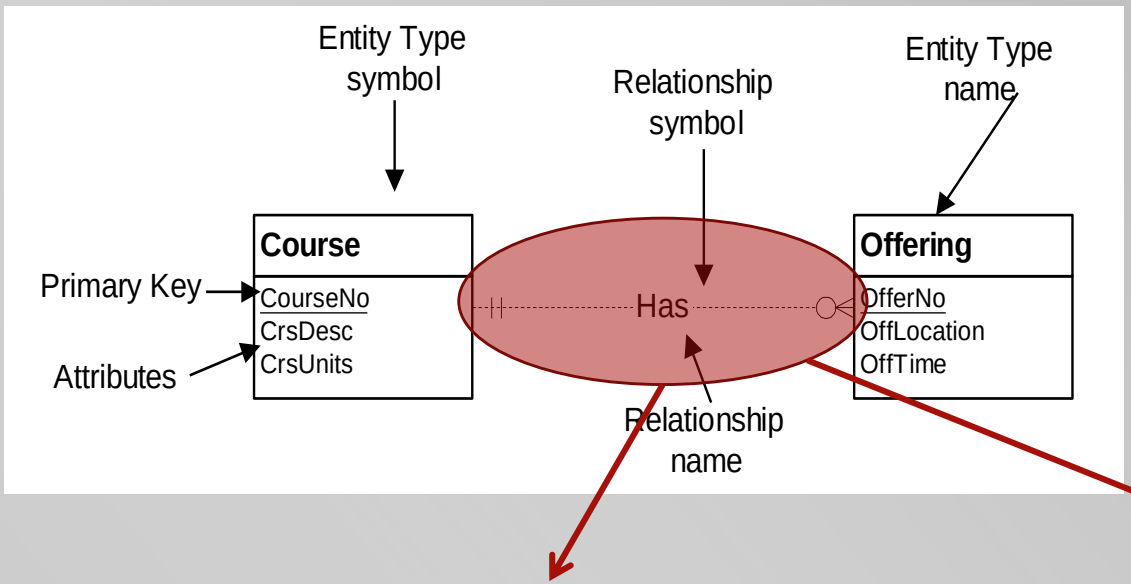
- Resumen notación modelo E/R (Chen)

Notation	Purpose
	represents entity types
	represents weak entity types
	represents relationship type
	represents identifying relationship
	represents attributes
	represents multivalued attribute
	represents key attribute

	represents partial key of weak entity type
	represents derived attribute
	connects attributes to entity types and entity types to relationship types
	represents total participation
	represents 1:1 relationship
	represents 1:M relationship
	represents M:1 relationship
	represents M:N relationship

Diseño conceptual

- Resumen notación modelo E/R (Crow's Foot)

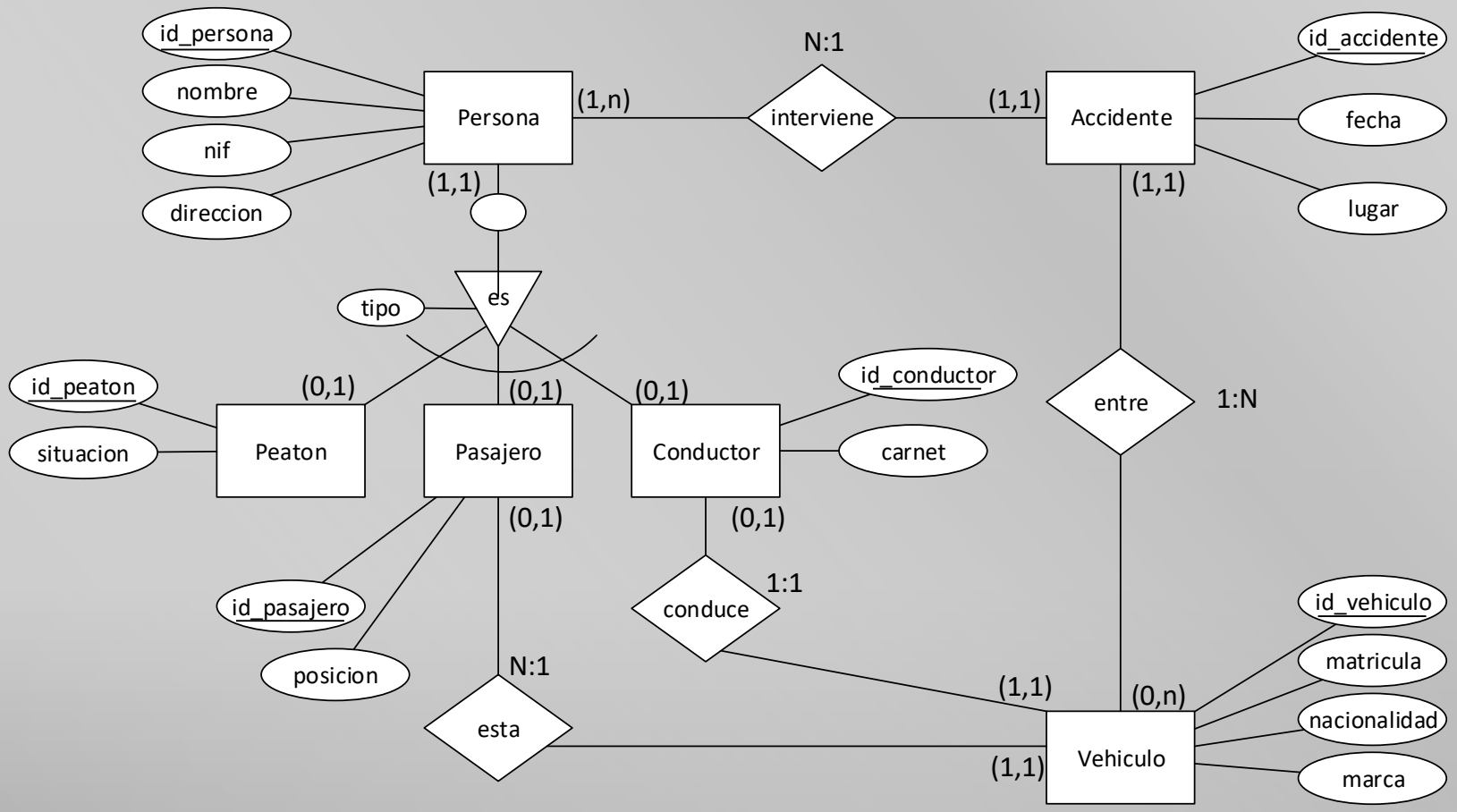


Symbol	Meaning
Dotted line	Non-identifying relationship
Solid line	Identifying relationship

Symbol	Meaning
	One—Mandatory
	Many—Mandatory
	One—Optional
	Many—Optional

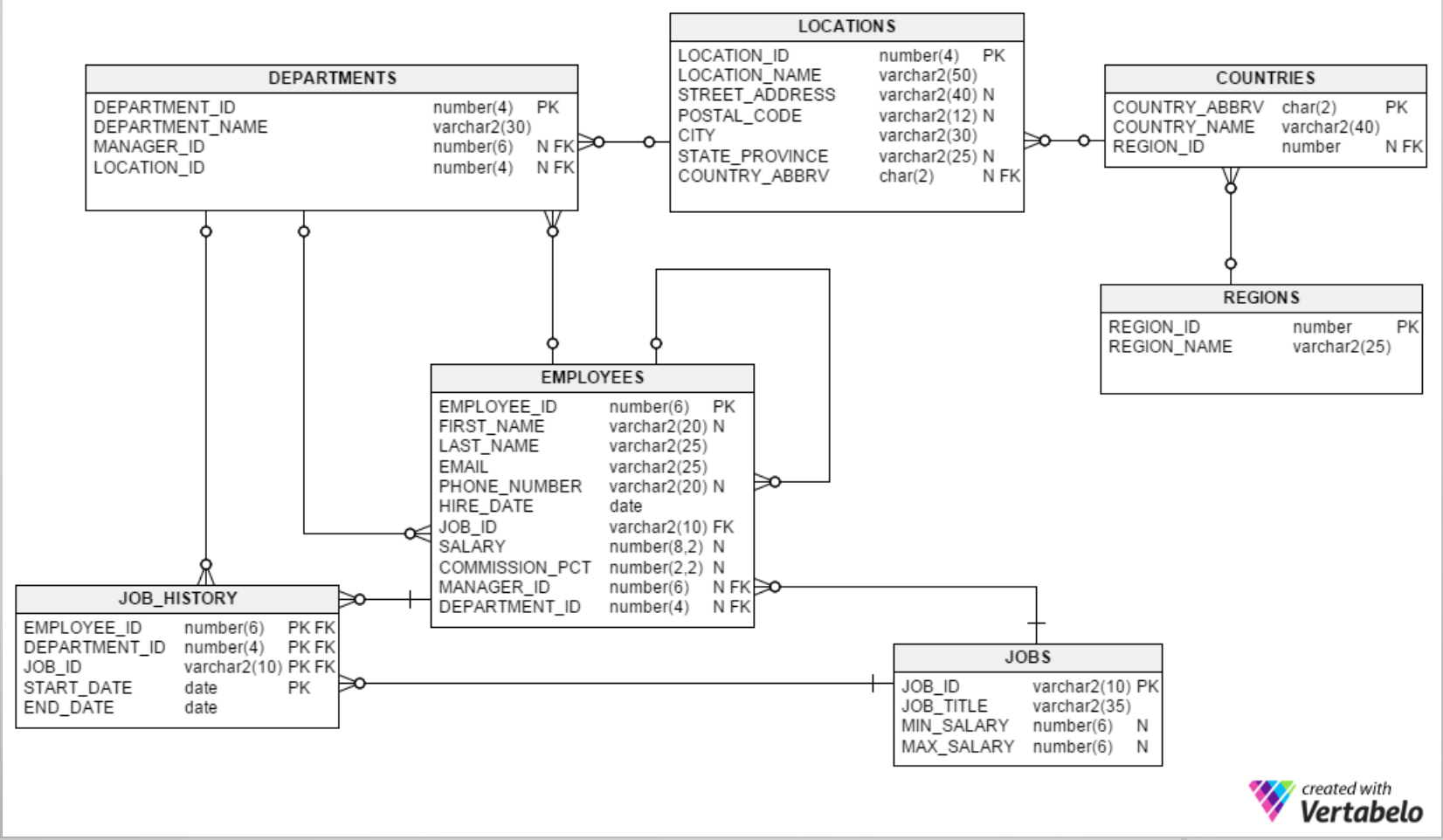
Diseño conceptual

- Ejemplo 1: Notación Chen



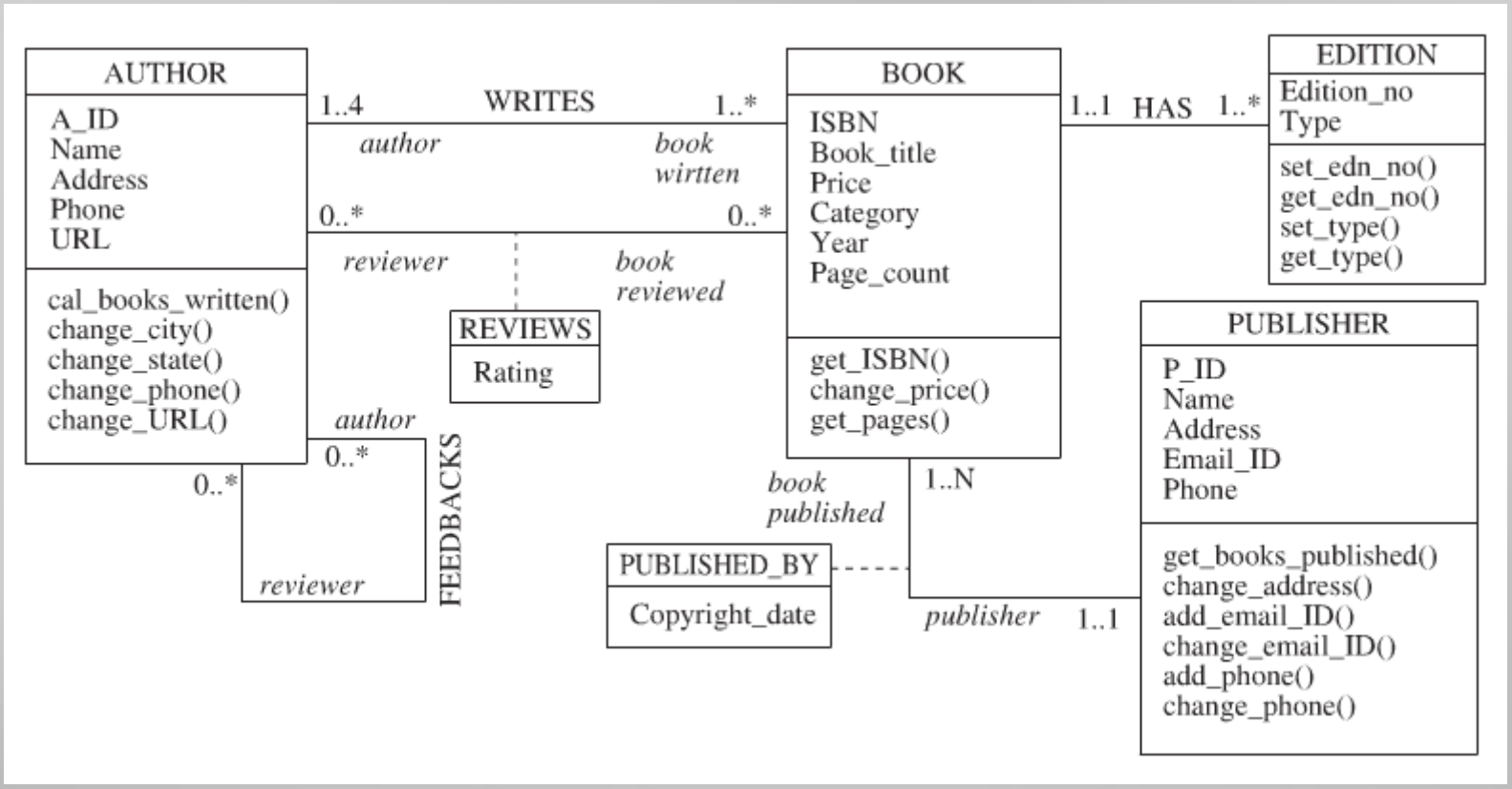
Diseño conceptual

- Ejemplo 2: Notación Crow's Foot



Diseño conceptual

- Ejemplo 3: Notación UML



Diseño lógico

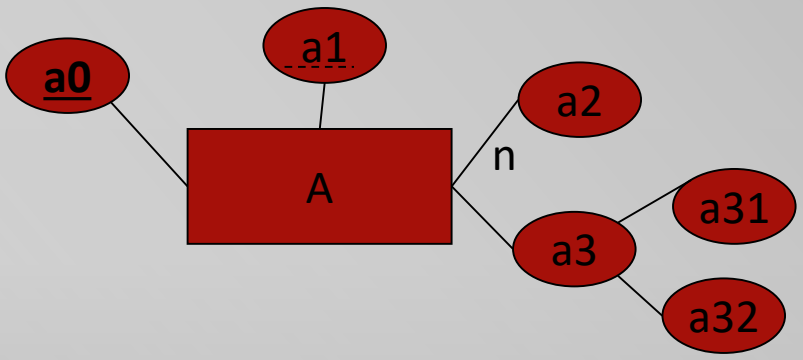
- El diseño lógico consiste en una transformación de modelos.
  - El modelo conceptual se convierte al modelo de la base de datos para el que se está diseñando, en nuestro caso modelo relacional
- Será necesario comprobar que el modelo lógico resultante garantiza las propiedades de eliminación de redundancias, accesibilidad sencilla de los datos, etc.
  - Para ello, en el caso de bases de datos relacionales, recurriremos a la teoría de la normalización, que nos permitirá obtener las mejores relaciones posibles para un esquema lógico.

Diseño lógico

- El paso de un esquema en el modelo E/R al relacional está basado en los tres principios siguientes:
  - Todo tipo de entidad se convierte en una relación.
  - Todo tipo de relación N:M se transforma en una relación.
  - Todo tipo de relación 1:N se traduce en el fenómeno de propagación de clave o se crea una nueva relación.

Diseño lógico

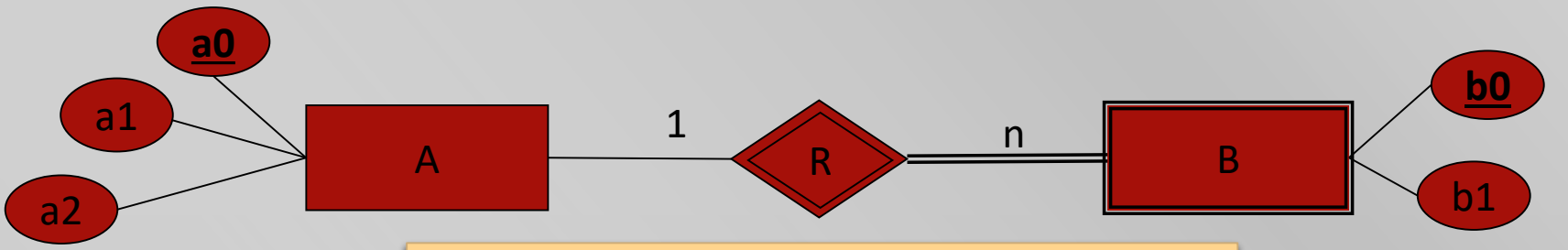
- Transformación de entidades
  - Entidades Fuertes o Regulares
    - Se crea una relación para cada entidad fuerte del esquema E/R.
    - Se incluyen todos los atributos que caracterizan las entidades.
    - De los atributos compuestos se incluyen sólo sus componentes.



A(a0:dom_a0,a1:dom_a1,a2:dom_a2,a31:dom_a31,a32:dom_a32)
Clave primaria: a0

Diseño lógico

- Transformación de entidades
  - Entidades Débiles
    - Se transforman como las entidades fuertes
    - Incorporan como atributos las PK de las entidades que identifican



A(a0:dom_a0,a1:dom_a1,a2:dom_a2……)
Clave primaria: a0

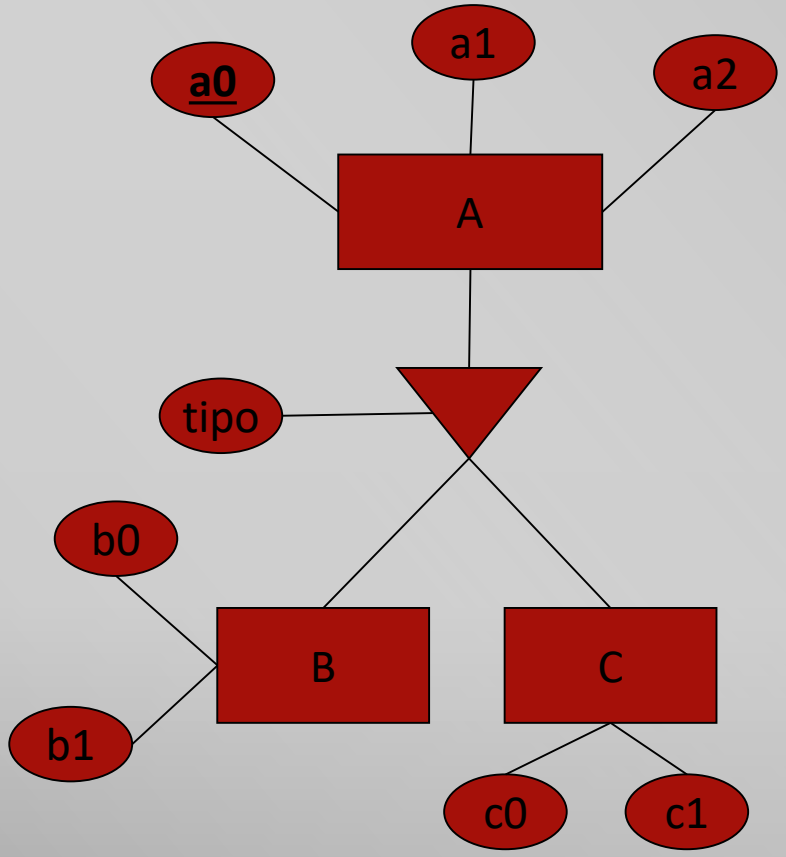
B(b0:dom_b0,b1:dom_b1,a0:dom_a0)
Clave Primaria: {a0,b0}
Clave Ajena:{a0} hace referencia a A

Diseño lógico

- Transformación de la herencia
  - Existen varias posibilidades:
    1. Crear una relación para el supertipo, y tantas relaciones como subtipos existan. Esta es la mejor opción desde el punto de vista semántico.
       - Las relaciones de los subtipos heredan la clave primaria de la relación del supertipo y además será una clave ajena.
    2. Crear sólo relaciones para los subtipos, añadiendo en cada una de ellas los atributos pertenecientes al supertipo.
    3. Integrar todas las entidades en una relación, incluyendo en ella los atributos del supertipo y de los subtipos, incluyendo el atributo discriminativo

Diseño lógico

- Transformación de la herencia



A (a0:dom_a0,a1:dom_a1,a2:dom_a2,
tipo:dom_tipo)
B (b0:dom_b0,b1:dom_b1,a0:dom_a0)
C (c0:dom_c0,c1:dom_c1,a0:dom_a0)

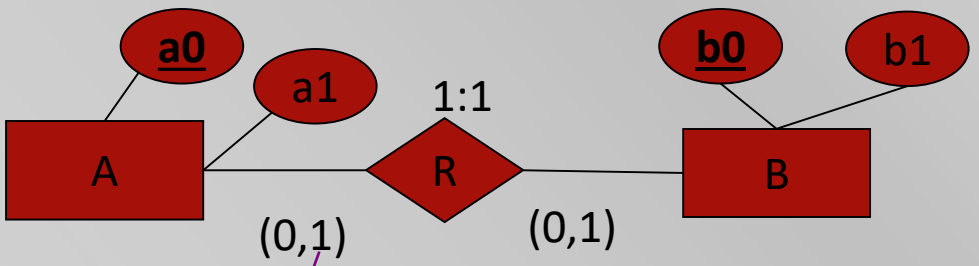
B (a0:dom_a0,a1:dom_a1,a2:dom_a2,
tipo:dom_tipo,b0:dom_b0,b1:dom_b1)
C (a0:dom_a0,a1:dom_a1,a2:dom_a2,
tipo:dom_tipo,c0:dom_c0,c1:dom_c1)

A (a0:dom_a0,a1:dom_a1,a2:dom_a2,
tipo:dom_tipo,b0:dom_b0,b1:dom_b1,
c0:dom_c0,c1:dom_c1)

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias 1:1, sin restricción de existencia

Se propaga la clave de una relación a la otra



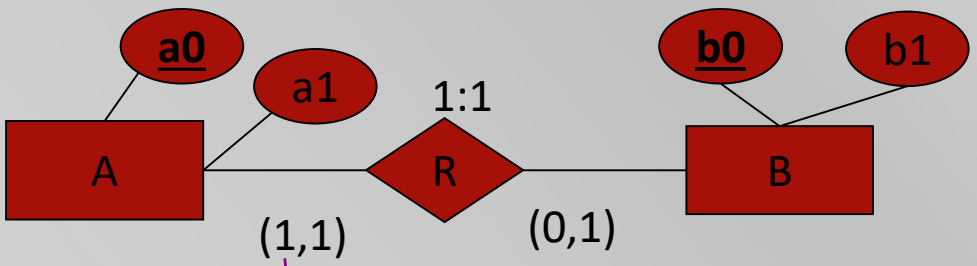
Puede ser recomendable crear una relación aparte si se prevé muchos nulos

A(a0:dom_a0,a1:dom_a1,…)
Clave Primaria: {a0}
B(b0:dom_b0,b1:dom_b1…..a0:dom_a0)
Clave Primaria:{b0}
Único: {a0}
Clave ajena:{a0} en A

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias 1:1, con restricción de existencia

Se propaga
la clave de una
relación a la otra

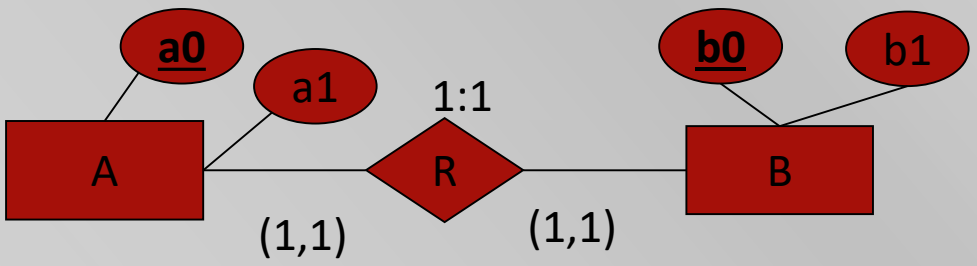


A(a0:dom_a0,a1:dom_a1,…)
Clave Primaria: {a0}
B(b0:dom_b0,b1:dom_b1…..a0:dom_a0)
Clave Primaria:{b0}
Único: {a0}
Valor no nulo: {a0}
Clave ajena:{a0} en A

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias 1:1, con dos restricciones de existencia

Se crea una relación con todos los atributos de las dos entidades



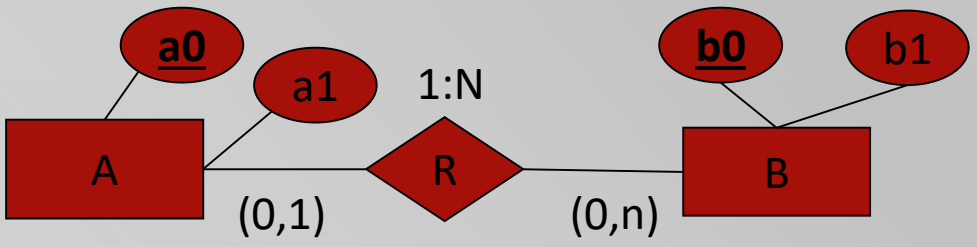
A-B (a0:dom_a0,a1:dom_a1,…b0:dom_b0…b1:dom_b1)
Clave Primaria: {a0}
Única: {b0}
Valor no nulo: {b0}

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias 1:N
    - Las dos entidades dan lugar a dos relaciones y la relación se representa con una clave ajena en la relación con mayor cardinalidad.
    - Para los casos de restricciones de existencia, se aplican los mismos principios que en el caso de las relaciones 1:1 con respecto a las restricciones de integridad.

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias 1:N



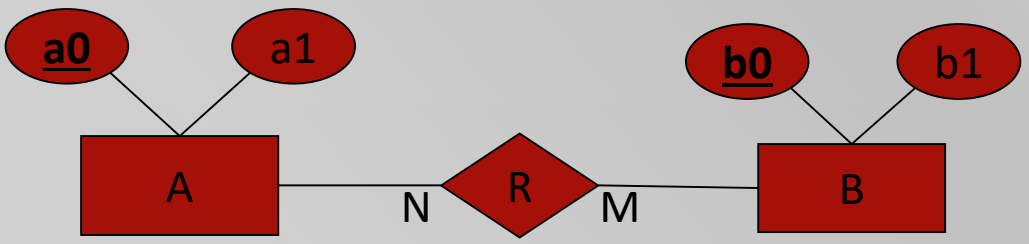
A(a0:dom_a0,a1:dom_a1,…)
Clave Primaria: {a0}
B(b0:dom_b0,b1:dom_b1…..a0:dom_a0)
Clave Primaria: {b0}
Clave ajena: {a0} en A

Diseño lógico

- Transformación de relaciones
  - Relaciones binarias N:M
    - Se transforman en una relación que tendrá como clave primaria la concatenación de los atributos principales de cada una de las entidades que relaciona.
    - Estos atributos deben ser clave ajena respecto a cada una de las tablas donde ese atributo es clave primaria.
    - Habrá que considerar lo que ocurre en los casos en los que se borre o modifique la clave primaria referenciada.

Diseño lógico

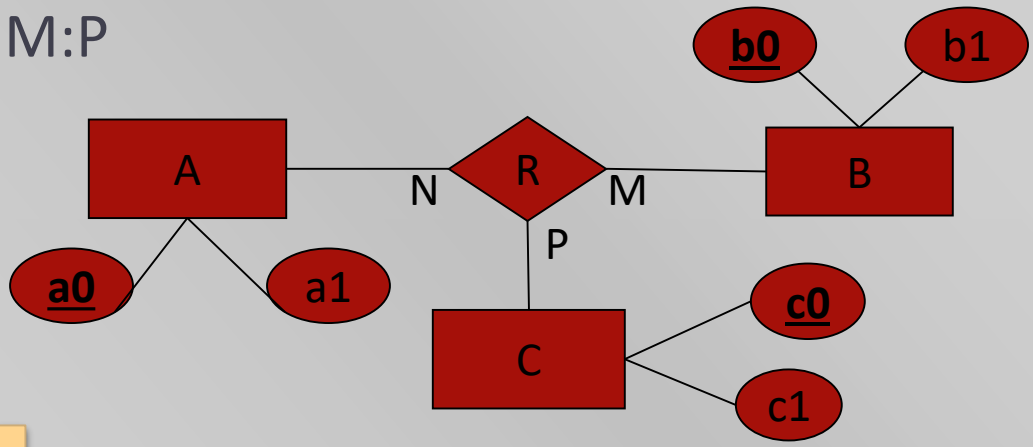
- Transformación de relaciones
  - Relaciones binarias N:M



A(a0:dom_a0,a1:dom_a1,…,an:dom_an)
Clave Primaria: {a0}
B(b0:dom_b0,b1:dom_b1…..)
Clave Primaria:{b0}
R(a0:dom_a0,b0:dom_b0)
Clave Primaria: {a0,b0}
Clave ajena:{a0} en A
Clave ajena:{b0} en B

Diseño lógico

- Transformación de relaciones
  - Relaciones ternarias N:M:P



A(a0:dom_a0,a1:dom_a1,…)
Clave Primaria: {a0}
B(b0:dom_b0,b1:dom_b1)
Clave Primaria:{b0}
C(c0:dom_c0,c1:dom_c1)
Clave Primaria:{c0}

R(a0:dom_a0,b0:dom_b0,c0:dom_c0)
Clave Primaria:{a0,b0,c0}
Clave Ajena: {a0} en A
Clave Ajena: {b0} en B
Clave Ajena: {c0} en C

Diseño lógico

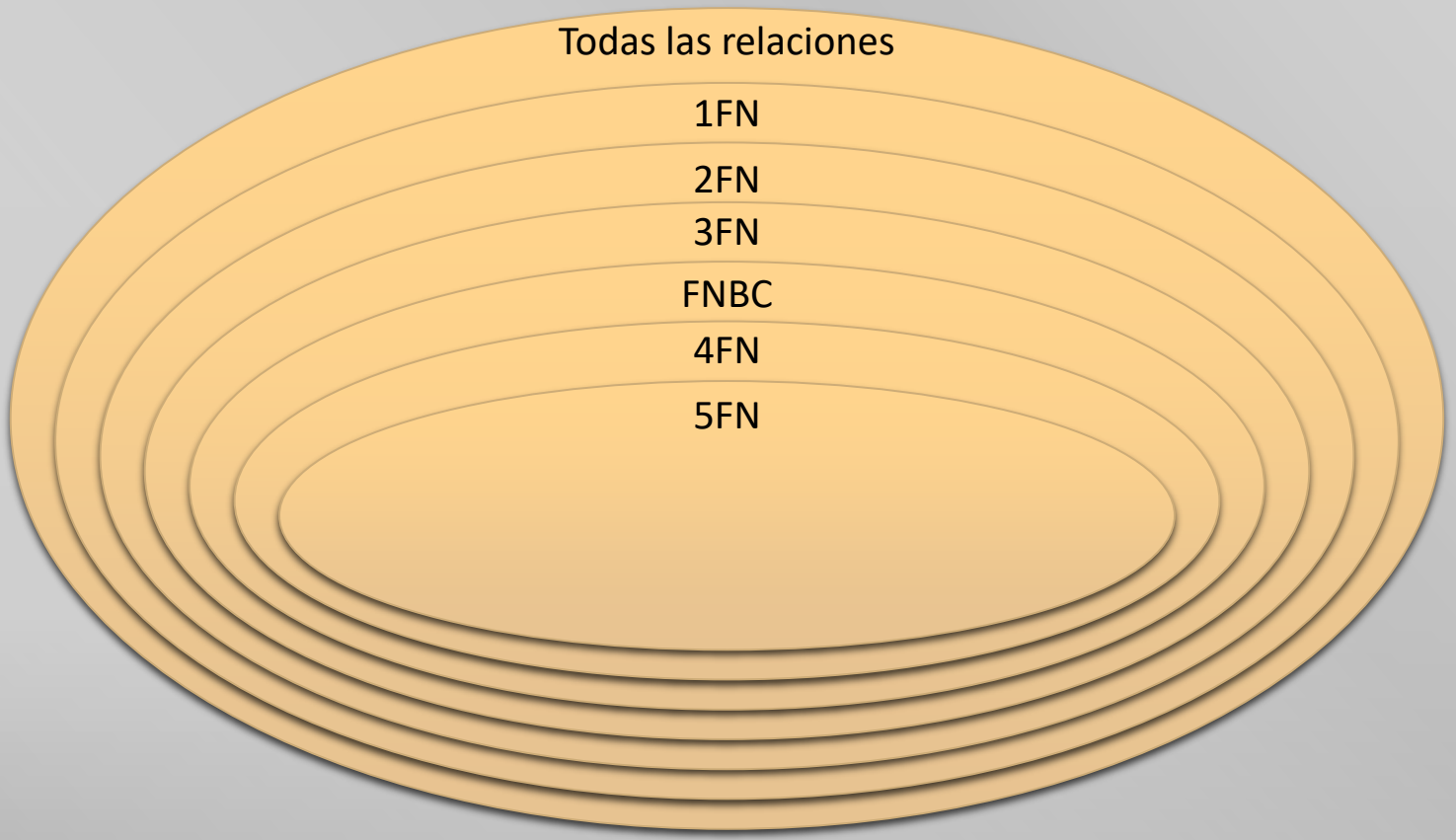
- Consideraciones adicionales
  - Relaciones binarias con atributos
    - 1:1 o 1:N.
      - Los atributos se pueden propagar junto con la clave primaria de la relación referida
      - Si los atributos son identificativos, se puede generar una relación nueva para la inter-relación
    - N:M. El atributo pasa a ser un atributo de la relación.
  - Relaciones binarias 1:1 o 1:N
    - Si existen muchos valores nulos, la propagación de clave puede ser ineficiente, y es más conveniente crear una nueva relación, el igual que en el caso N:M.

Normalización

- La teoría de la normalización nos permite reconocer cuando una relación tiene propiedades indeseables y nos muestra como convertirlas a formas más deseables.
- La teoría de la normalización se basa en el concepto de formas normales:
  - Se dice que una relación está en una determinada forma normal si satisface un cierto conjunto de restricciones.
  - 1FN, 2FN, 3FN, FNBC,(4FN y 5FN).

Normalización

- Formas Normales

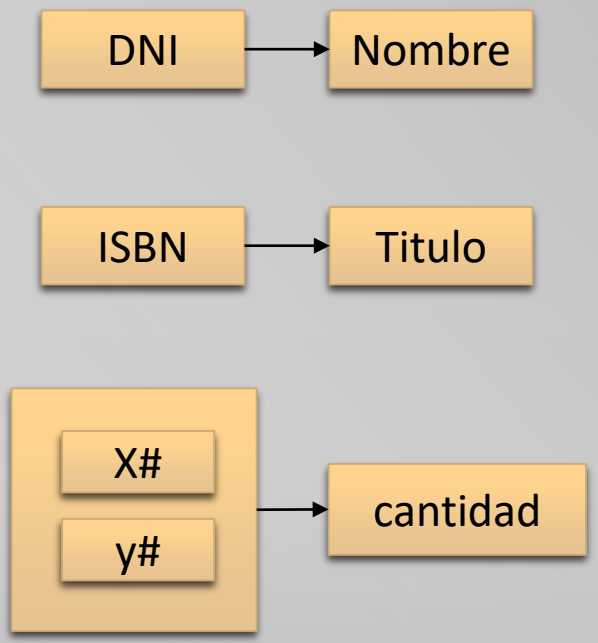


Normalización

- Dependencia Funcional
  - Dada una relación R, el atributo Y de R depende funcionalmente del atributo X de R (R.X->R.Y) si y sólo si un valor de Y en R está asociado a cada valor de X en R (en cualquier momento dado).
  - Dada una relación R, el atributo Y de R depende funcionalmente del atributo X de R si y sólo si siempre que dos tuplas de R concuerden en su valor de X, deben por fuerza concordar en su valor de Y.
- Dependencia Funcional Completa
  - Se dice que el atributo Y de la relación R es por completo dependiente funcionalmente del atributo X de R si depende funcionalmente de X y no depende funcionalmente de ningún subconjunto propio de X.

Normalización

- Dependencias Funcionales
  - Representación



Normalización

- Primera, segunda y tercera forma normal
  - Una relación está en 1FN si y sólo si todos los dominios simples subyacentes contienen sólo valores atómicos.
  - Una relación está en 2FN si y sólo si está en 1FN y todos los atributos no clave tienen dependencia funcional completa de la clave primaria.
  - Una relación está en 3FN si y sólo está en 2FN y todos los atributos no clave dependen de manera no transitiva de la clave primaria. Es decir, si los atributos no clave son:
    - mutuamente independientes y
    - y tienen dependencia funcional completa de la clave primaria

Normalización

- Ejemplo 1FN

vcod	nombre	**telefonos**	dir
v1	pepe	**963547876,963456765**	Moliner,5,Valencia
v2	juan	**965453453,963467655**	M. Hernandez,3,Valencia



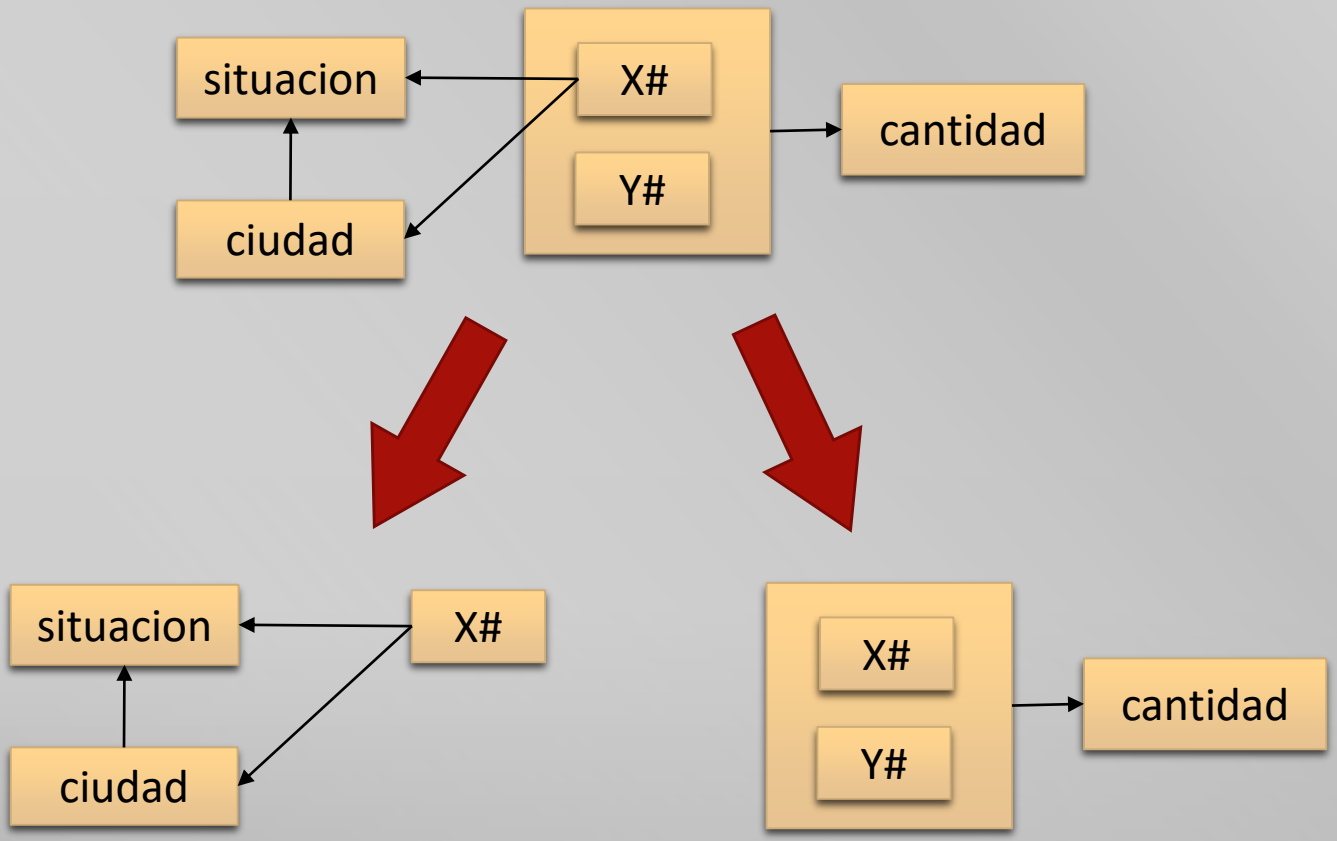


vcod	nombre	calle	numero	ciudad
v1	pepe	Moliner	5	Valencia
v2	juan	M.Hernandez	3	Valencia

vcod	telefonos
v1	963547876
v1	963456765
v2	965453453
v2	963467655

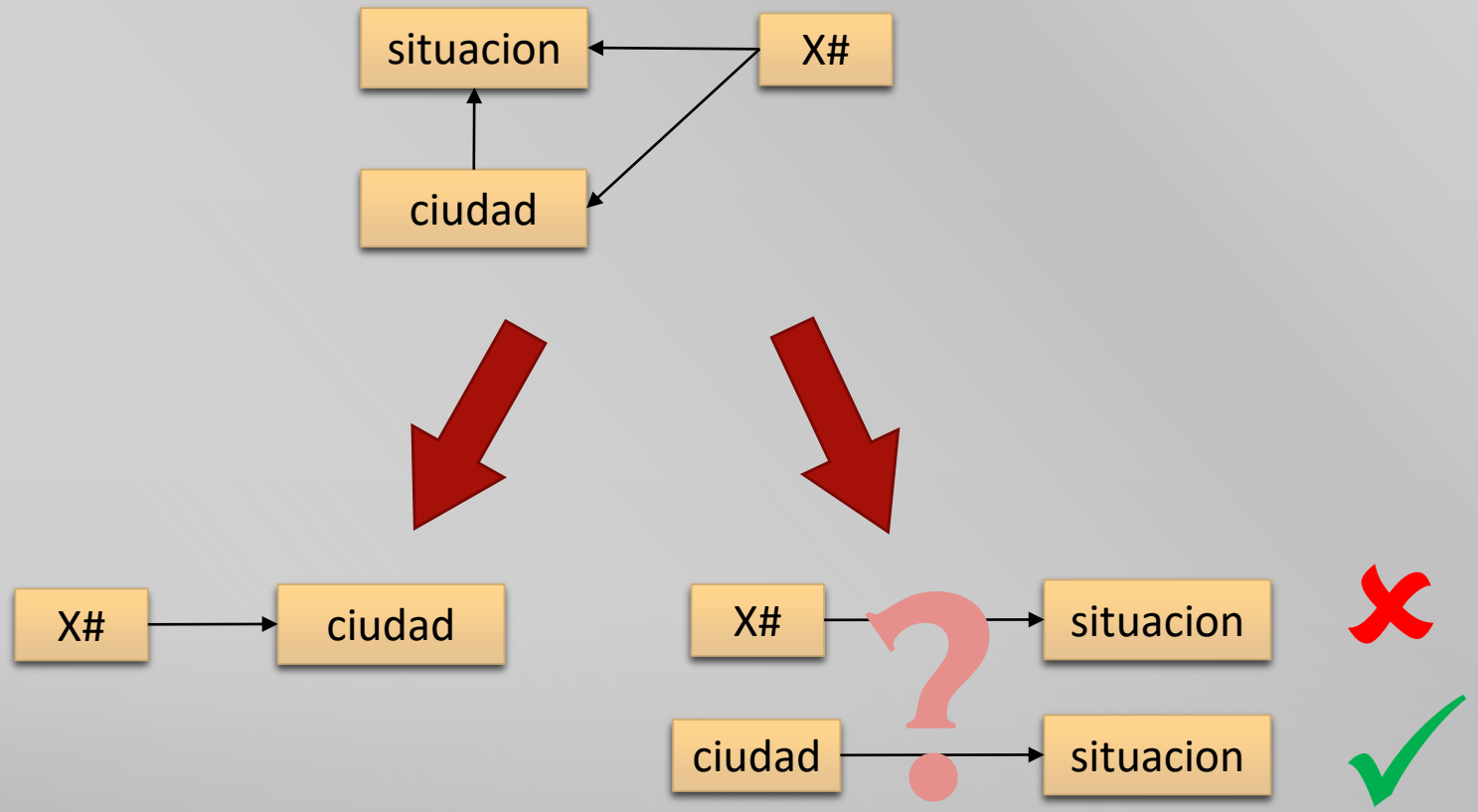
Normalización

- Ejemplo 2FN



Normalización

- Ejemplo 3FN



Normalización

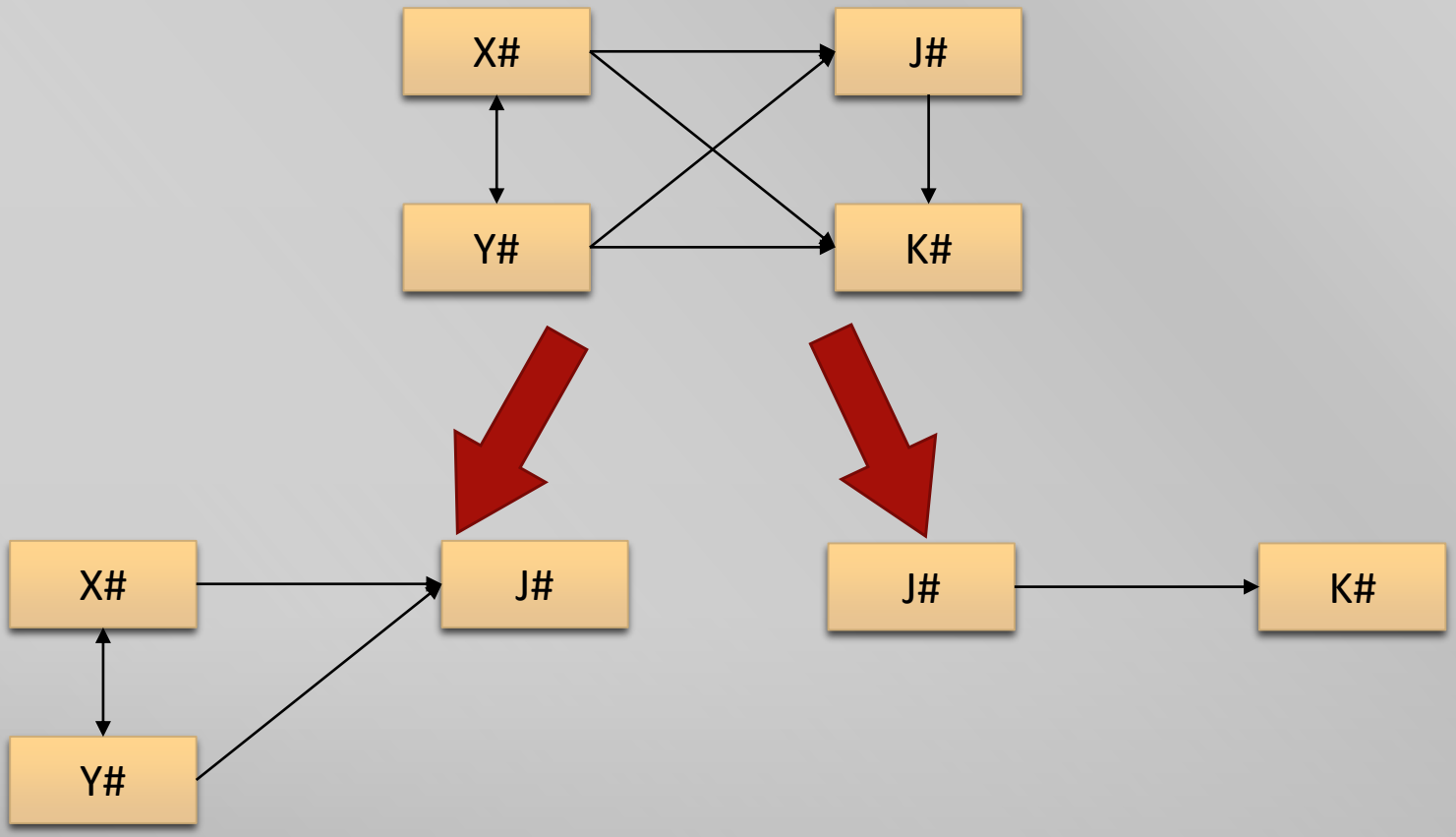
- Forma Normal de Boyce/Codd (FNBC)
  - La 3FN no maneja relaciones que:
    - Tiene varias claves candidatas,
    - Las claves candidatas son compuestas,
    - y las claves candidatas se solapan, (tienen por lo menos un atributo en común).
  - Se define la 3 FNBC para el caso que existan más de una clave candidata.
  - Concepto de determinante:
    - atributo del cual depende funcionalmente algún otro atributo.

Normalización

- Forma Normal de Boyce/Codd (FNBC)
  - Una relación está en FNBC si y sólo todo determinante es una clave candidata.
  - En caso en que no se den las condiciones anteriores o no exista más de una clave candidata la FNBC es equivalente a la 3FN.
  - La definición implica:
    - que los únicos determinantes son las claves candidatas
    - o sea las flechas del diagrama de dependencias funcionales sólo salen de claves candidatas.

Normalización

- Forma Normal de Boyce/Codd (FNBC)



Diseño Físico

- El objetivo es mejorar el rendimiento.
- Se necesita conocer la carga que el sistema debe soportar y los requisitos de los usuarios que utilizaran la BD.
- La descripción de la carga y de los requisitos de los usuarios sobre el rendimiento de las consultas son la base para la toma de decisiones en un buen diseño lógico.

Diseño Físico

- Descripción de la carga del sistema:
  - Lista de consultas y sus frecuencias.
  - Lista de modificaciones y sus frecuencias.
  - Objetivos de rendimiento para cada tipo de consulta y modificación.
- Para cada consulta identificar:
  - las relaciones a las que se accede.
  - los atributos en la cláusula SELECT
  - los atributos en las condiciones (WHERE), y el grado de selectividad que pueden aportar (muy restrictivas, poca, etc).
- Para las modificaciones se debe conocer:
  - atributos en las condiciones (WHERE) y su grado de selectividad.
  - tipo de modificación (insert, delete, update).
  - Caso del update, campos que se actualizan.

Diseño Físico

- Tener en cuenta que actualizar campos con índices puede hacer más lento el proceso de actualización.
- Decisiones que hay que tomar:
  - Qué índices crear
  - Qué relaciones y campos de las relaciones se indexarán
  - Tipo de índice a utilizar
- Cuándo realizar cambios en el esquema conceptual para mejorar el rendimiento:
  - Desnormalización
  - Particionado
  - Vistas

Diseño Físico

- ¿Cuándo indexar?
  - No se deben crear índices al menos que alguna consulta se vaya a beneficiar de ello.
- Elección de la clave de búsqueda:
  - Los atributos que aparecen en la cláusula WHERE son los candidatos a recibir índices
    - Los atributos que se usan con criterio de búsqueda exacta pueden ser indexados con índices tipo hash.
    - Los atributos que se usan con criterio de búsqueda por rango pueden ser indexados utilizando Arboles B.

Diseño Físico

- Clave de búsqueda de múltiples atributos
  - Se deben considerar índices compuestos (de más de un atributo) cuando la cláusula WHERE incluye condiciones sobre más de un atributo de una relación.
  - Cuando se creen índices de este tipo, si existen condiciones de búsqueda en rangos, hay que tener cuidado con el orden de los atributos definido en el índice.
- Contemplar el coste del mantenimiento de índices:
  - Si el mantenimiento del índice ralentiza operaciones de actualización muy frecuentes, habrá que considerar la eliminación del índice.
  - Considerar que un índice puede acelerar el proceso de actualización, cuando el índice se define sobre los campos que se utilizan como criterio de selección de aquellas tuplas que se van a actualizar.